

ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
Rai Technology University
(Estd. under Karnataka Act No. 40 of 2013)

ASSIGNMENT [REPORT]

Subject Title: DEEP LEARNING

Subject Code: OCS351T

Submitted By:

Student Name: HARSHA SHREE P

University Enrolment Number: RTU24101CS051

Program Name: B.TECH CSE (AIML)

Semester: 5TH Sem

Session: 2026-2027

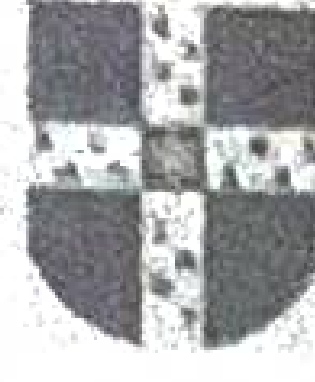
Submitted To:

Faculty Name: Mr. KUMAR SATYAM TANTI

Department of Computer Science & Engineering College

Of Engineering

Rai Technology University



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
Rai Technology University

(Estd. under Karnataka Act No. 40 of 2013)

ASSIGNMENT [REPORT]

Subject Title: DEEP LEARNING

Subject Code: OCS351T

Submitted By:

Student Name: HARSHA SHREE P

University Enrolment Number: RTU24101CS051

Program Name: B.TECH CSE (AIML)

Semester: 5TH Sem

Session: 2026-2027

Submitted To:

Faculty Name: Mr. KUMAR SATYAM TANTI

Department of Computer Science & Engineering College

Of Engineering

Rai Technology University

MNIST Handwritten Digit Classification

REPORT

ABSTRACT

Handwritten digit recognition is one of the most common applications of deep learning and computer vision. In this experiment, a neural network was developed using tensorflow/Keras to classify handwritten digits from the MNIST dataset. The dataset contains grayscale images of handwritten digits ranging from 0 to 9. The images were preprocessed, normalized and used to train a feedforward neural network. The performance of the model was evaluated on test data and the learning process was visualized using accuracy and loss graphs. An experimental study was also conducted by modifying the network architecture using a dropout rate and Deep learning.

layer. The model achieved high classification accuracy, demonstrating the effectiveness of neural networks in image recognition tasks.

Objective

- Load and explore the MNIST handwritten digit dataset. Display sample digit images. Design and train a neural network model. Evaluate model performance on test data. Visualize training and validation accuracy and loss. Test the model on five sample digits.

THEORY

MNIST Dataset

MNIST [Modified National Institute of Standards and Technology] is one of the most widely used benchmark datasets in machine learning and Deep learning.

The dataset consists of:

60,000 training images

10,000 testing images

Grayscale images

Image size 28×28 pixels

Classes: Digits 0-9.

Each image contains a handwritten digit represented by pixel intensity values ranging from 0 to 255.

Neural Network

A neural network is a computational model inspired by the human brain. It consists of interconnected layers of neurons.

The model used in this experiment contains:

(i) Input layer

(ii) Hidden Dense layer

(iii) Output layer

The hidden layer uses the ReLU activation function, while the output layer uses the softmax activation function for multi-class classification.

ACTIVATION FUNCTIONS

ReLU [Rectified linear unit]

$$f(x) = \max(0, x)$$

Advantages:

- Computationally efficient
- Reduces vanishing gradient problem.

Softmax

Converts output values into probabilities

$$p(y_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

The digit with the highest probability is selected as the predicted class.

Software & libraries used.

→ python

→ numpy

→ Google Colab

→ matplotlib

→ Tensorflow

→ Keras

METHODOLOGY

- 1 Dataset loading
- 2 Data preprocessing
- 3 Build Neural network
- 4 Compile Model
5. Train model
6. Evaluate model
7. Visualization

Code :

Import Libraries

```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np
```

Load MNIST Dataset

```
(x_train, y_train), (x_test, y_test) = keras.datasets.
mnist.load_data()
```

```
print("Training images:", x_train.shape)
```

```
print("Training labels:", y_train.shape)
```

```
print("Testing images:", x_test.shape)
```

```
print("Testing labels:", y_test.shape)
```

```
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
```

Display Sample images.

```
plt.figure(figsize=(10, 5))
```

```
for i in range(10):
```

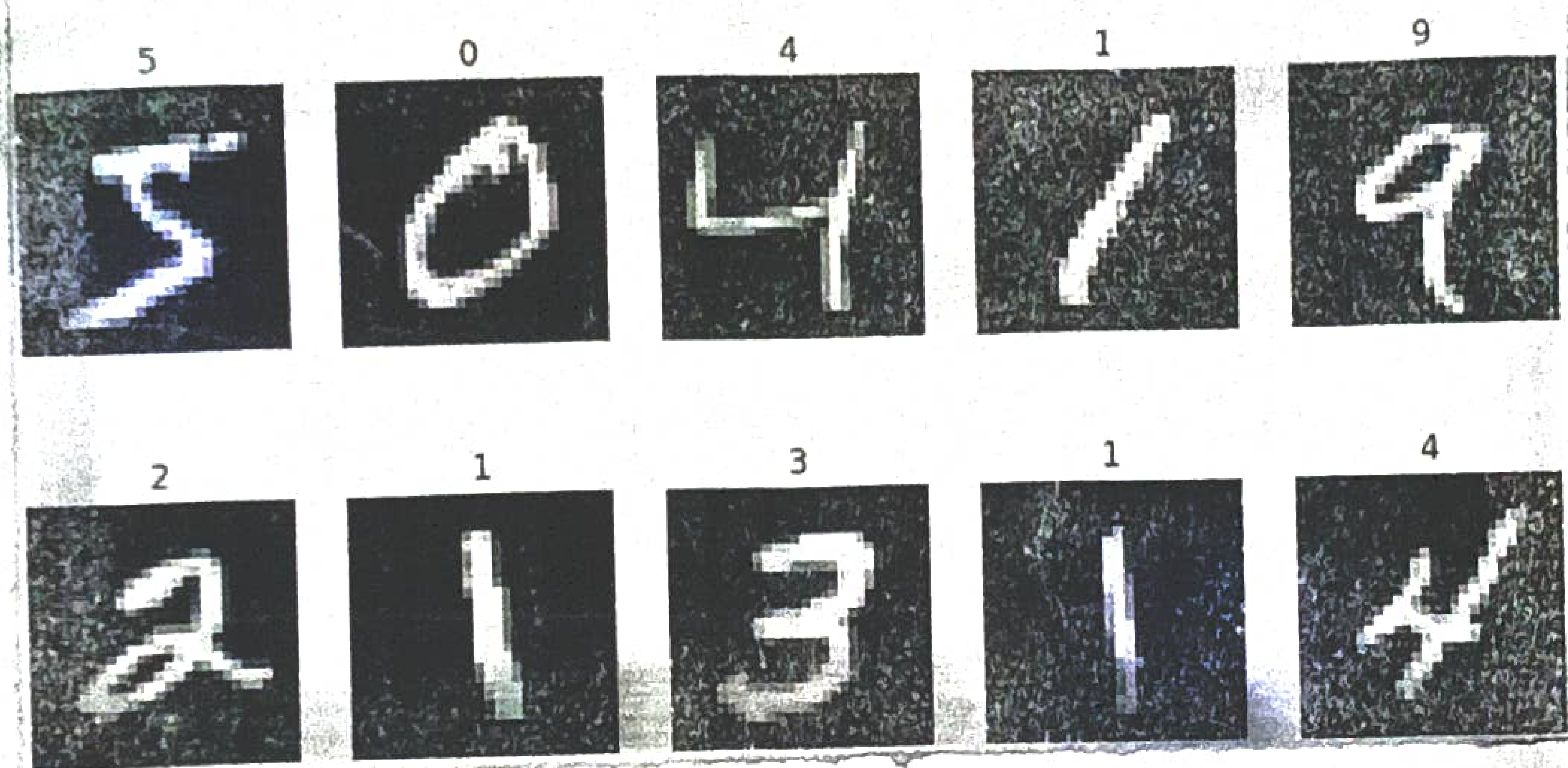
```
    plt.subplot(2, 5, i+1)
```

```
plt.imshow(x_train[i], cmap = 'gray').
```

```
plt.title(y_train[i])
```

```
plt.axis('off')
```

```
plt.show()
```



Normalize Data

```
x_train = x_train / 255.0
```

```
y_test = x_test / 255.0
```

Build Neural Network

```
model = keras.Sequential([
```

```
keras.layers.Flatten(input_shape = (28, 28)),
```

```
keras.layers.Dense(128, activation = 'relu'),
```

```
keras.layers.Dense(10, activation = 'softmax').
```

```
])
```

```
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 784)	0
dense_4 (Dense)	(None, 128)	100,480
dense_5 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

Compile model.

model.compile

optimizer = 'adam',

loss = 'sparse_categorical_crossentropy',

metrics = ['accuracy']

)

Train the model.

history = model.fit

x_brain,

y_brain,

epochs = 10,

validation_split = 0.2

)

```
Epoch 1/10
1500/1500 — 8s 5ms/step - accuracy: 0.9178 - loss: 0.2854 - val_accuracy: 0.9529 - val_loss: 0.1021
Epoch 2/10
1500/1500 — 6s 4ms/step - accuracy: 0.9630 - loss: 0.1276 - val_accuracy: 0.9627 - val_loss: 0.1243
Epoch 3/10
1500/1500 — 8s 5ms/step - accuracy: 0.9741 - loss: 0.0880 - val_accuracy: 0.9706 - val_loss: 0.1004
Epoch 4/10
1500/1500 — 7s 4ms/step - accuracy: 0.9808 - loss: 0.0653 - val_accuracy: 0.9701 - val_loss: 0.1010
Epoch 5/10
1500/1500 — 7s 5ms/step - accuracy: 0.9847 - loss: 0.0510 - val_accuracy: 0.9733 - val_loss: 0.0931
Epoch 6/10
1500/1500 — 7s 5ms/step - accuracy: 0.9878 - loss: 0.0403 - val_accuracy: 0.9751 - val_loss: 0.0891
Epoch 7/10
1500/1500 — 6s 4ms/step - accuracy: 0.9905 - loss: 0.0317 - val_accuracy: 0.9752 - val_loss: 0.0863
Epoch 8/10
1500/1500 — 6s 4ms/step - accuracy: 0.9923 - loss: 0.0249 - val_accuracy: 0.9734 - val_loss: 0.0868
Epoch 9/10
1500/1500 — 6s 4ms/step - accuracy: 0.9941 - loss: 0.0199 - val_accuracy: 0.9716 - val_loss: 0.1078
Epoch 10/10
1500/1500 — 6s 4ms/step - accuracy: 0.9946 - loss: 0.0187 - val_accuracy: 0.9742 - val_loss: 0.1018
```

Evaluate accuracy

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
```

```
print("Test Accuracy = ", test_accuracy)
```

313/313 — 1s 3ms/step - accuracy: 0.9765 - loss: 0.0826
Test Accuracy = 0.9764999747276386

plot Accuracy Graph

```
plt.plot(history.history['accuracy'])
```

```
plt.plot(history.history['val-accuracy'])
```

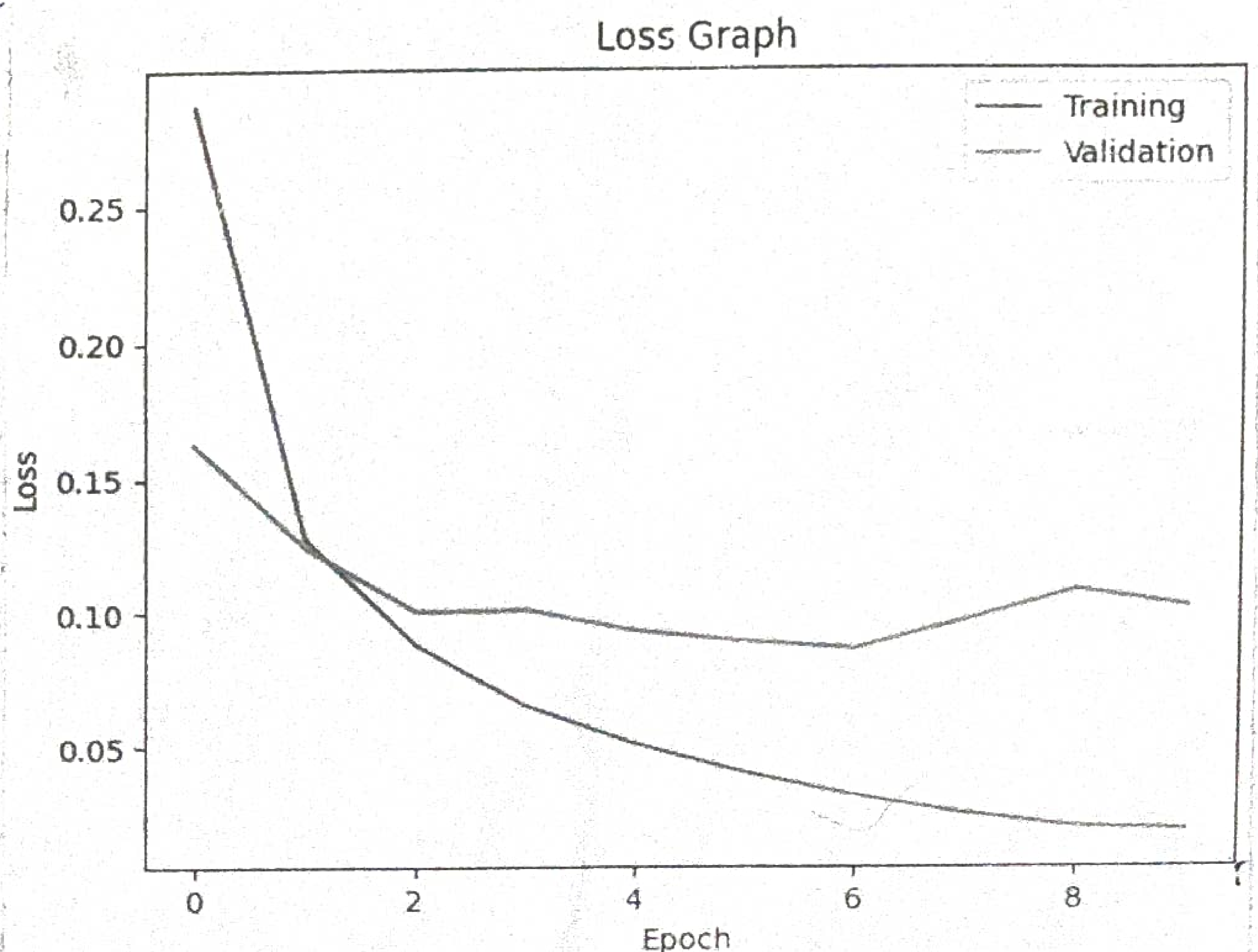
```
plt.title("Accuracy Graph")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Accuracy")
```

```
plt.legend(['Training', 'Validation'])
```

```
plt.show()
```



Plot Loss Graph.

```
plt.plot(history.history['loss'])
```

```
plt.plot(history.history['val-loss'])
```

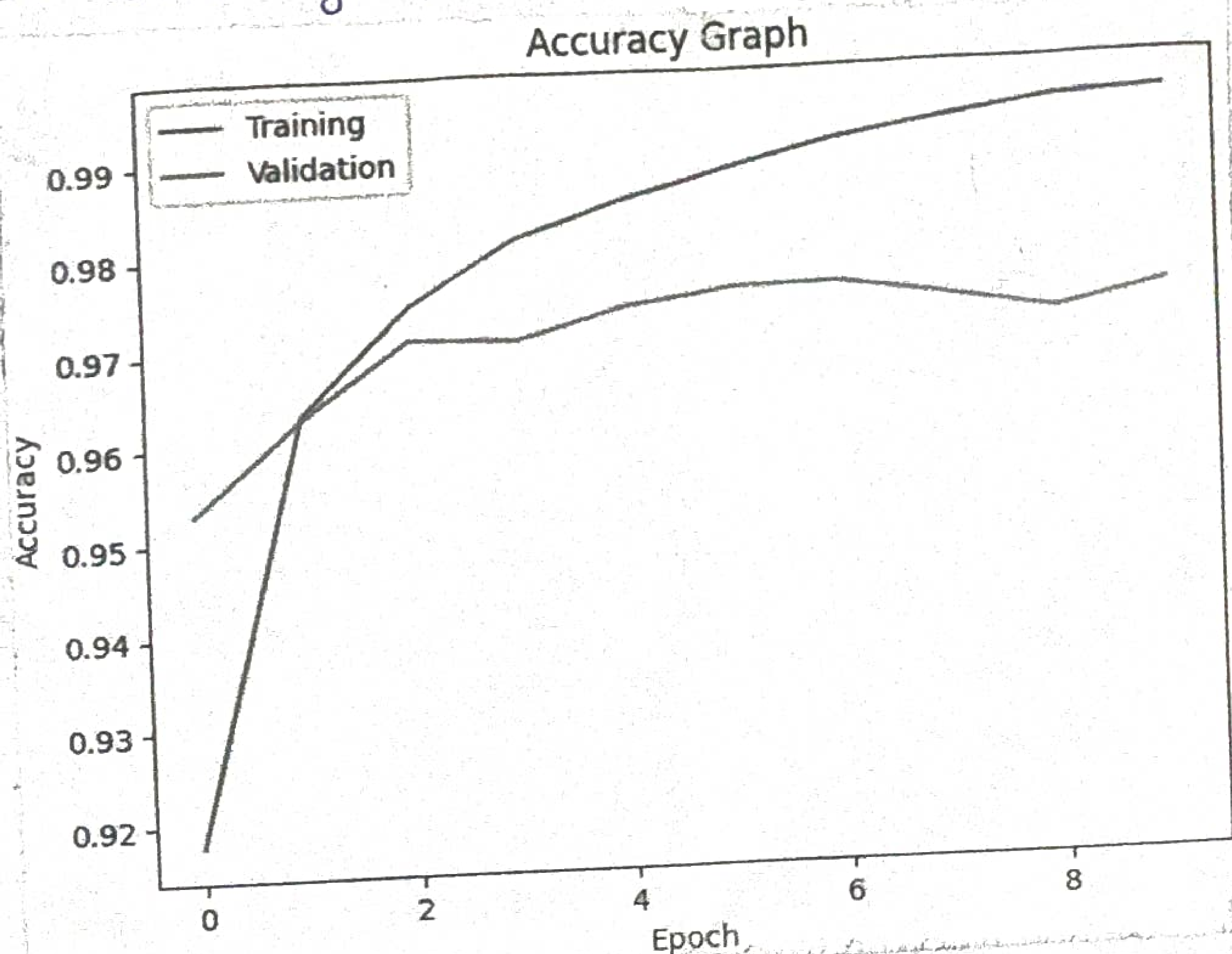
```
plt.title("loss graph")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.legend(['Training', 'Validation'])
```

```
plt.show()
```



Test on 5 images.

```
predictions = model.predict(X_test[:5])
```

```
for i in range(5):
```

```
    predicted_label = np.argmax(predictions[i])
```

```
    plt.imshow(X_test[i], cmap='gray')
```

```
    plt.title()
```

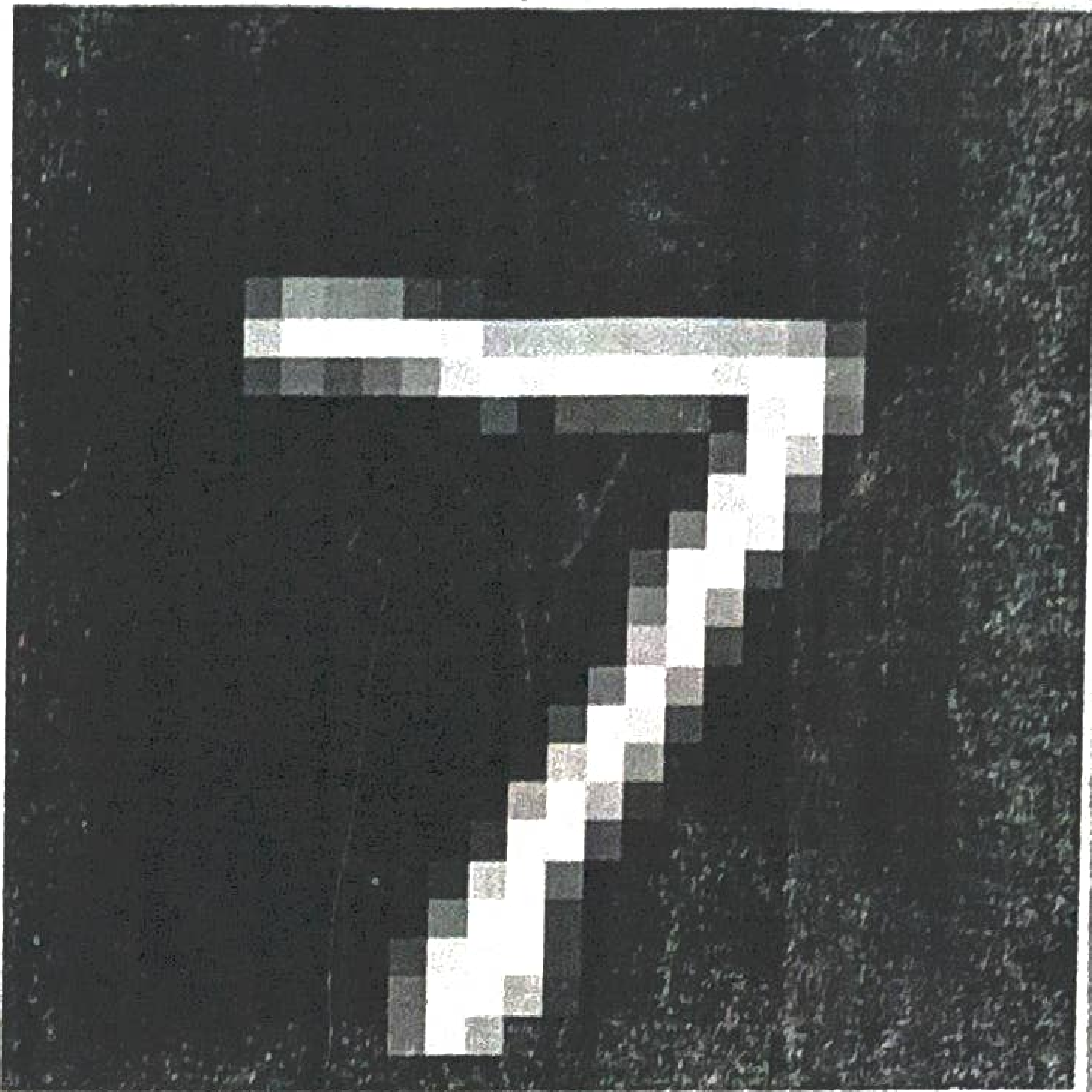
if "Actual: {y-test[i]} | Predicted: {predicted-label}"

```
)  
plt.axis('off')  
plt.show()
```

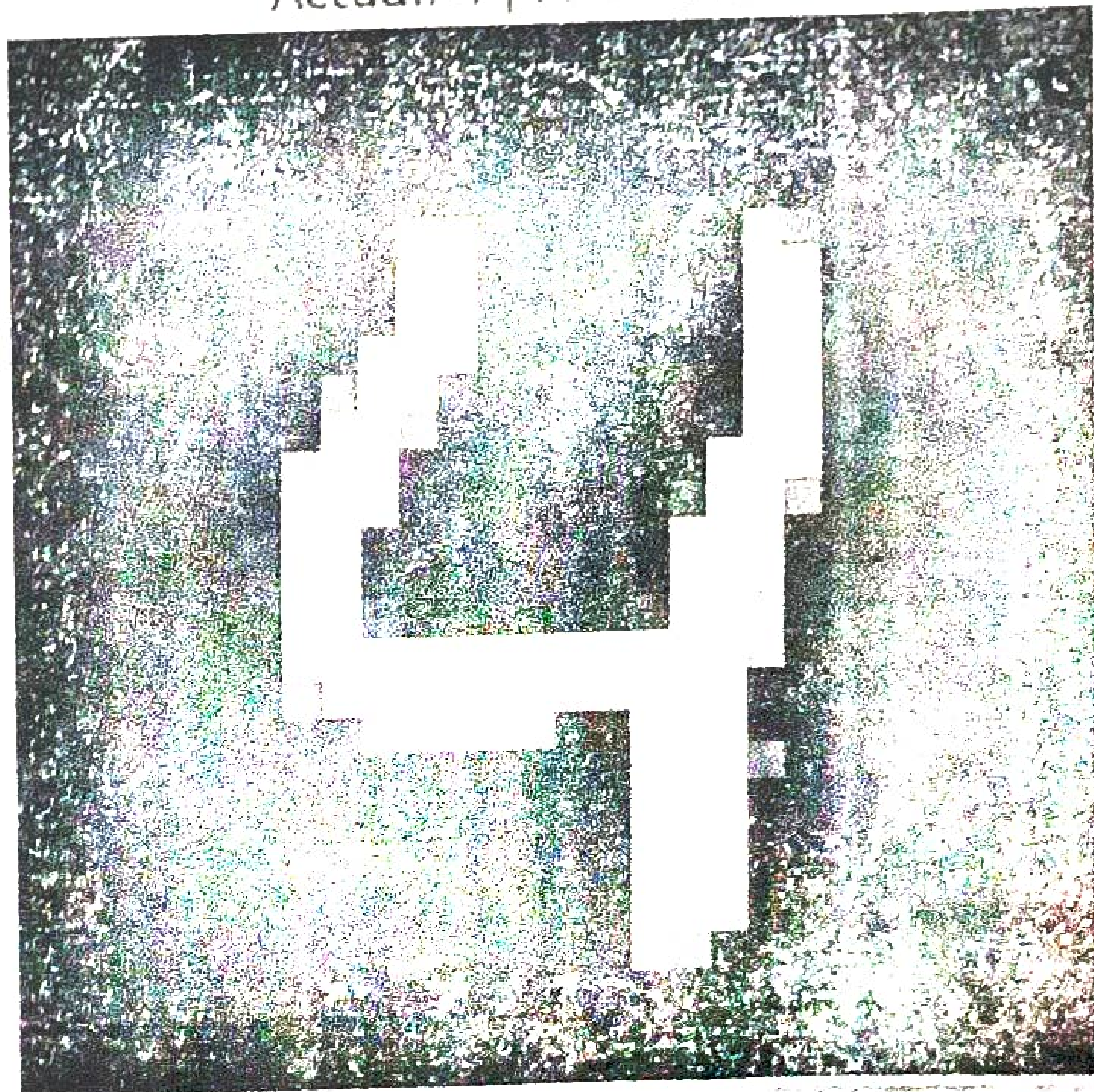
1/1

0s 65ms/step

Actual: 7 | Predicted: 7



Actual: 4 | Predicted: 4



```
# Create comparison table  
print("Actual | Predicted")
```

```
for i in range(5):
```

```
    pred = np.argmax(predictions[i])
```

```
    print(y_test[i], "|t", pred)
```

Actual	Predicted
7	7
2	2
1	1
0	0
4	4

```
# Perform experiment
```

```
# Add dropout layer
```

```
# Create a new model:
```

```
experiment_model.compile(  
    optimizer = 'adam',
```

```
    loss = 'sparse_categorical_crossentropy',
```

```
    metrics = ['accuracy']
```

```
)
```

```
# Train
```

```
history2 = experiment_model.fit(  
    x_train,
```

```
    y_train,
```

```
    epochs = 10,
```

```
    validation_split = 0.2)
```

```

Epoch 1/10
1500/1500 ----- 8s 5ms/step - accuracy: 0.8985 - loss: 0.3486 - val_accuracy: 0.9545 - val_loss: 0.1606
Epoch 2/10
1500/1500 ----- 7s 5ms/step - accuracy: 0.9483 - loss: 0.1757 - val_accuracy: 0.9653 - val_loss: 0.1197
Epoch 3/10
1500/1500 ----- 10s 5ms/step - accuracy: 0.9580 - loss: 0.1369 - val_accuracy: 0.9705 - val_loss: 0.1042
Epoch 4/10
1500/1500 ----- 6s 4ms/step - accuracy: 0.9663 - loss: 0.1104 - val_accuracy: 0.9722 - val_loss: 0.0932
Epoch 5/10
1500/1500 ----- 10s 4ms/step - accuracy: 0.9703 - loss: 0.0979 - val_accuracy: 0.9731 - val_loss: 0.0875
Epoch 6/10
1500/1500 ----- 7s 5ms/step - accuracy: 0.9729 - loss: 0.0846 - val_accuracy: 0.9751 - val_loss: 0.0878
Epoch 7/10
1500/1500 ----- 6s 4ms/step - accuracy: 0.9757 - loss: 0.0755 - val_accuracy: 0.9760 - val_loss: 0.0823
Epoch 8/10
1500/1500 ----- 7s 5ms/step - accuracy: 0.9779 - loss: 0.0699 - val_accuracy: 0.9775 - val_loss: 0.0797
Epoch 9/10
1500/1500 ----- 6s 4ms/step - accuracy: 0.9776 - loss: 0.0688 - val_accuracy: 0.9760 - val_loss: 0.0850
Epoch 10/10
1500/1500 ----- 11s 5ms/step - accuracy: 0.9792 - loss: 0.0617 - val_accuracy: 0.9778 - val_loss: 0.0814

```

Evaluate:

```

loss 2, acc 2 = experiment - model.evaluate(X-test, y-test)
print("Experiment Accuracy = ", acc 2)

```

```

313/313 ----- 1s 3ms/step - accuracy: 0.9796 - loss: 0.0735
Experiment Accuracy = 0.9796000123023987

```

Compare results

```

print("Original Model Accuracy = ", test-accuracy)
print("Experiment model Accuracy = ", acc 2)

```

```

Original Model Accuracy = 0.9764999747276306
Experiment Model Accuracy = 0.9796000123023987

```